أسئلة الجزء الأول

ليه لما بتعرف Custom Constructor ده بيلغي الـ Default Constructor في C#؟

لما بتعمل Custom Constructor، الكومبايلر بيفهم إنك مش عايز أي Object يتكريت إلا بشروطك وبالمعطيات اللي إنت محددها. عشان كده بيلغي الـ Default Constructor التلقائي (اللي من غير Parameters) عشان يمنع إن الـ Object يتعمله تهيئة (Initialization) بشكل ناقص أو غير متوافق مع اللوجيك بتاعك.

إزاي الـ Method Overloading بيحسن قراءة الكود وإعادة استخدامه؟

بيخليك تقدر تستخدم نفس اسم الدالة (زي Sum) عشان تعمل عمليات منطقية متشابهة لكن بأنواع أو عدد بيانات مختلف. ده بيخلي الكود أنضف وأسهل في القراءة بدل ما تضطر تخترع اسم جديد لكل دالة، وبكده بتوحد المسميات للوظائف المتشابهة.

إيه الهدف من الـ Constructor Chaining في الوراثة؟

هدفه ضمان إن الكلاس الأب يتعمله تهيئة صحيحة للخصائص بتاعته قبل ما الكلاس الابن يبدأ يهيئ الخصائص الإضافية الخاصة بيه. ده بيمنع تكرار الكود لأنك بتعيد استخدام كود التهيئة اللي مكتوب أصلاً في الأب.

إيه الفرق بين new و override في الـ Method Overriding؟

كلمة override بتعدل الدالة الأصلية الموجودة في الأب. ولما تستدعيها، النسخة الخاصة بالابن هي اللي هتتنفذ دايماً، حتى لو كنت بتتعامل مع الـ Object عن طريق Reference من نوع الأب.

أما كلمة new بتخفي دالة الأب وتعمل دالة جديدة تماماً ومستقلة. لو استدعيتها من خلال Reference للأب، هينفذ نسخة الأب، ولو من خلال Reference للابن، هينفذ نسخة الابن.

ليه دايماً بنعمل Override للـ ToString() في الكلاسات بتاعتنا؟

لأنها بشكلها الافتراضي بترجع اسم الكلاس فقط. لما بنعملها Override، بنقدر نخليها ترجع نص مفيد ومقروء يعبر عن حالة الكائن الحالية والبيانات اللي جواه.

ليه مينفعش تاخد Instance (كائن) من Interface بشكل مباشر؟

لأن الـ Interface عبارة عن عقد أو تصميم بيحتوي على أسماء الدوال بس من غير أي كود فعلي لتنفيذها. فلو تم إنشاء كائن منه، مش هيكون عنده أي منطق لتنفيذ الدوال دي.

إيه فوايد الـ Default Implementations في الـ Interfaces اللي ظهرت في C# 8.0؟

الفائدة الرئيسية هي التوافقية. بتسمحلك تضيف دالة جديدة للـ Interface وتحطيلها كود افتراضي من غير ما تعطل الكلاسات القديمة اللي بتعمل Implement للـ Interface ده، لأنها هتستخدم الكود الافتراضي لو مكتبتش كود خاص بيها.

ليه من المفيد استخدام Interface Reference عشان نوصل لدوال الكلاس اللي بينفذه؟

ده بيحقق مبدأ الارتباط الضعيف. يعني بتكتب كود بيعتمد على العقد نفسه مش على كلاس محدد. ده بيسمحلك تبدل الكلاسات ببعضها بسهولة طالما إنها بتنفذ نفس الـ Interface.

إزاي C# بتتغلب على قيود الوراثة المفردة باستخدام الـ Interfaces؟

لغة C# بتمنع إن كلاس يورث من أكتر من كلاس واحد عشان تتجنب مشاكل التضارب، لكنها بتسمح للكلاس إنه يعمل Implement لعدد لا نهائي من الـ Interfaces. ده بيدي للكلاس القدرة على اكتساب خصائص متعددة.

إيه الفرق بين الـ Virtual Method والـ Abstract Method؟

الـ virtual بيكون ليها كود أساسي مبدئي في الكلاس الأب، والكلاس الابن مخير إنه يسيب الكود زي ما هو أو يعمل له Override.

أما الـ abstract عبارة عن اسم وتوقيع بس من غير أي كود في الكلاس الأب، والكلاس الابن مجبر إنه يكتب كودها ويعملها Override.

أسئلة الجزء الثاني

إيه الفرق بين الـ class والـ struct في C#؟

الـ class يعتبر Reference Type (بيتخزن في الـ Heap)، بيدعم الوراثة، ولما بتساويه بمتغير تاني بياخد مرجع لنفس المكان.

الـ struct يعتبر Value Type (بيتخزن في الـ Stack)، لا يدعم الوراثة (إلا الـ Interfaces)، ولما بتساويه بمتغير تاني بياخد نسخة مستقلة من البيانات.

لو الوراثة علاقة بين الكلاسات، وضح العلاقات التانية الممكنة:

بجانب الوراثة (Is-A)، فيه علاقات تانية أهمها:

علاقة Association وهي علاقة استخدام عامة بين كلاسين.

علاقة Aggregation (Has-A) وهي علاقة احتواء ضعيفة؛ كلاس بيحتوي على كلاس تاني بس الجزء يقدر يعيش لوحده.

علاقة Composition وهي علاقة احتواء قوية؛ كلاس بيحتوي على كلاس تاني ولا يمكن للجزء إنه يعيش بدون الكل.

علاقة Dependency وهي علاقة اعتماد مؤقتة.

أسئلة الجزء الثالث الإضافي

Self Study Report:

ده مجرد تقرير نظري بسيط بتكتب فيه بخطوات سريعة إيه اللي اتعلمته من الـ Task ده، والمشاكل اللي واجهتك وحليتها إزاي.

إيه هو الـ Static Binding والـ Dynamic Binding؟

الـ Static Binding بيحصل وقت الـ Compile-time. الكومبايلر بيكون عارف مسبقاً وبدقة أنهي دالة هتتنفذ. من أشهر أمثلته الـ Method Overloading.

أما الـ Dynamic Binding بيحصل وقت الـ Run-time. تحديد الدالة اللي هتتنفذ بيتم بناءً على النوع الفعلي للكائن. من أشهر أمثلته الـ Method Overriding.